

Two-Phase Locking (2PL) — Complete Study Notes

Growing Phase · Shrinking Phase · Lock Point · Basic / Conservative / Strong Strict
Deadlocks · Cascading Aborts · Wait-Die · Wound-Wait · Wait-For Graph

TOPICS COVERED

1. What is 2PL? Growing & Shrinking Phases

2. Lock Point & Serializability

3. Basic 2PL — Problems

4. Conservative 2PL — No Deadlocks

5. Strong Strict 2PL — Industry Standard

6. Cascading Aborts — Cause & Fix

7. Deadlock Prevention: Timeout, WFG

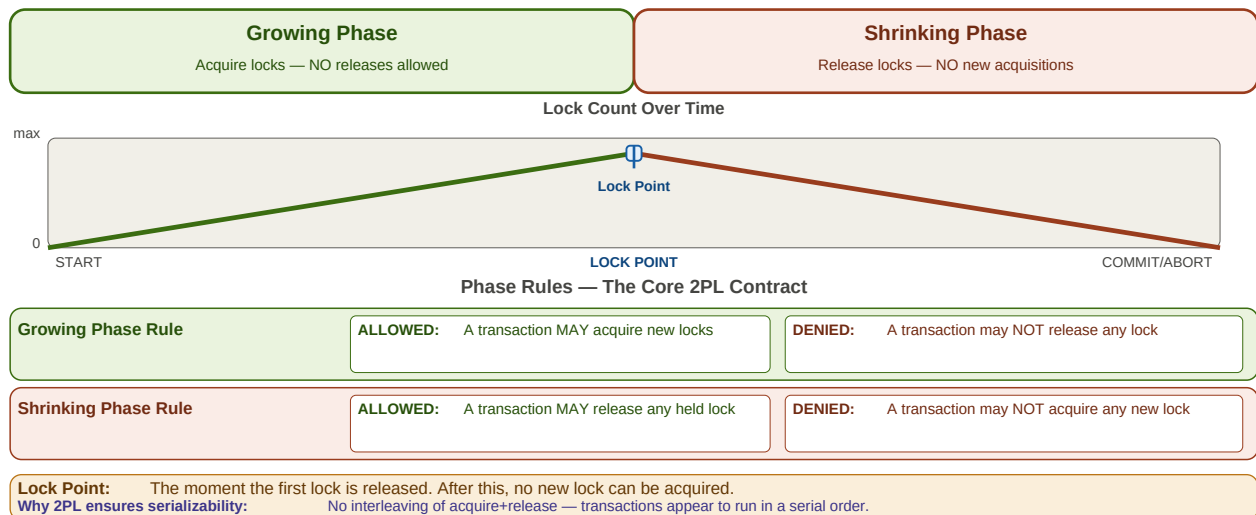
8. Timestamp Schemes: Wait-Die, Wound-Wait

1. What is Two-Phase Locking (2PL)?	3
2. Growing Phase & Shrinking Phase	3
3. Lock Point & Why 2PL Ensures Serializability	3
4. Three Variants — Basic, Conservative, Strong Strict	4
5. Basic 2PL — Deadlocks & Cascading Aborts	5
6. Cascading Abort — Cause, Example & Fix	5
7. Deadlock Detection & Prevention	6
8. Timestamp Schemes — Wait-Die & Wound-Wait	6
9. Money Transfer Example — All Variants Compared	7
10. Quick Revision & Interview Questions	7

1, 2 & 3. What is 2PL? — Phases & Serializability

Two-Phase Locking (2PL) is a **pessimistic concurrency control protocol** that enforces serializability by splitting every transaction into exactly two phases: a growing phase (acquire locks only) and a shrinking phase (release locks only). Once a transaction releases its first lock, it can never acquire a new one.

Two-Phase Locking — Growing Phase vs Shrinking Phase



Growing phase: lock count rises. Lock Point = peak. Shrinking phase: lock count falls. No reacquire after first release.

Phase	Action allowed	Action denied	Effect
Growing Phase	Acquire S-locks and X-locks freely	Release ANY lock	Lock set monotonically increases
Shrinking Phase	Release any held lock	Acquire ANY new lock	Lock set monotonically decreases
Lock Point	Peak of lock count — transition point	N/A	Defines serialization order of transactions

Why 2PL ensures serializability: No transaction can interleave acquire-and-release cycles with another. The ordering of lock points across transactions defines a serial schedule.

Lock Point: the moment the FIRST lock is released. All transactions can be ordered by their lock points — this order IS the serial execution order.

Important: 2PL only guarantees serializability — NOT freedom from deadlocks or cascading aborts.

4. Three Variants of Two-Phase Locking

2PL comes in three flavours, each trading off between concurrency, deadlock prevention, and cascading abort prevention. Understanding which variant to use and why is a frequent interview topic.

Three Variants of Two-Phase Locking

Basic 2PL Locks released anytime during shrinking phase Lock Acquire: During growing phase Lock Release: Gradual — anytime in shrinking Concurrency: Higher Deadlocks: YES Cascading Aborts: YES Verdict: Rarely used in production too many problems Lock timeline: Acquire Release Commit	Conservative 2PL (Static) ALL locks acquired BEFORE transaction begins Lock Acquire: ALL at start — before any read/write Lock Release: Gradual after execution Concurrency: Lower Deadlocks: NO Cascading Aborts: YES Verdict: Rarely used too restrictive, hard to predict all locks Lock timeline: ALL Execute (all locks held) Release gradually	Strong Strict 2PL (Rigorous) ALL locks held until COMMIT or ABORT Lock Acquire: During growing phase Lock Release: ONLY at commit/abort Concurrency: Medium Deadlocks: YES Cascading Aborts: NO Verdict: Industry standard best consistency vs concurrency balance Lock timeline: Acquire Hold till commit Rel
---	--	--

Basic: flexible but problematic · Conservative: no deadlocks but low concurrency · Strong Strict: industry standard

Variant	Acquire When	Release When	Deadlocks	Cascading Aborts	Concurrency	Used In
Basic 2PL	Growing phase	Anytime in shrinking	YES	YES	High	Rarely — too many problems
Conservative 2PL	ALL at start, before execution	Gradually after execution	NO	YES	Low	Rarely — hard to predict all locks
Strong Strict 2PL	Growing phase	ONLY at commit/abort	YES	NO	Medium	Industry standard — InnoDB, PostgreSQL

5 & 6. Cascading Aborts — The Core Basic 2PL Problem

Cascading abort happens when Transaction T1 releases a lock after modifying data but before committing. T2 reads T1's uncommitted data ("dirty read"). If T1 then aborts, T2 must also abort — and anything that read T2 must abort too. This cascade can be arbitrarily long.

Cascading Abort — Basic 2PL Problem Strong Strict Fix

Basic 2PL — Cascading Abort	Strong Strict 2PL — No Cascading
Lock released before commit = dirty read risk	All locks held until commit/abort
T1 Writes A=90, holds X-lock	T1 Writes A=90, holds X-lock
T1 Releases X-lock (shrinking phase)	T1 Still holds X-lock (no early release)
T2 Reads A=90 (dirty! T1 not committed)	T2 Tries to read A — BLOCKED by T1 lock
T2 Commits using A=90	T1 COMMITS — releases all locks
T1 ABORTS — A should revert to 100	T2 Now reads A=90 (committed data only)
T2 MUST ALSO ABORT — used dirty data!	T2 Commits — consistent state guaranteed
Problem: T2 committed based on uncommitted data = cascading abort chain. Expensive to resolve.	Fix: T2 only reads committed data = no cascade Tradeoff: T2 blocked longer. Deadlocks still possible.
Rule: Basic 2PL releases locks early → dirty reads → cascading aborts. Strong Strict 2PL: Never release before commit → no dirty reads → no cascading aborts. Deadlocks still possible. Conservative 2PL: Pre-acquires all locks → no deadlocks → cascading aborts possible (releases before commit).	

Basic 2PL: early lock release = dirty reads = cascading aborts · Strong Strict: hold till commit = no dirty reads = no cascade

Money transfer example: T1 debits A (100->90), releases lock. T2 reads A=90 and uses it. T1 aborts — A reverts to 100. T2 must abort too: it committed based on A=90 which no longer exists.

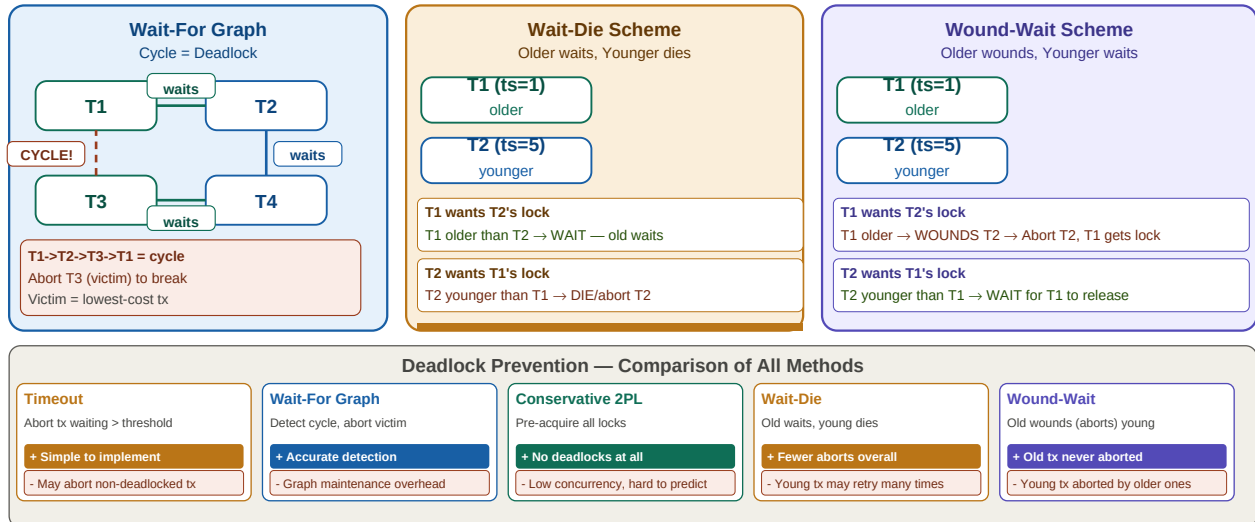
Strong Strict 2PL fix: T1 holds X-lock on A until commit. T2 is BLOCKED trying to read A. T1 commits, lock released. T2 now reads A=90 (committed). No cascade possible.

Conservative 2PL does NOT fix cascading aborts — it still releases locks before commit. It only fixes deadlocks.

7 & 8. Deadlock Detection, Prevention & Timestamp Schemes

Deadlocks arise in Basic and Strong Strict 2PL when transactions form a circular wait. Four approaches exist: detect after the fact (Timeout, Wait-For Graph), prevent entirely (Conservative 2PL), or use timestamp ordering to break ties before they form (Wait-Die, Wound-Wait).

Deadlock Handling — Wait-For Graph, Wait-Die, Wound-Wait



Wait-For Graph: cycle=deadlock, abort victim · Wait-Die: old waits, young dies · Wound-Wait: old wounds young · 5 methods compared

Method	How it works	Resolves Deadlock	Prevents Deadlock	Drawback
Timeout	Abort tx waiting longer than threshold	YES (after fact)	NO	May abort non-deadlocked txns
Wait-For Graph	Build graph, detect cycle, abort victim	YES (after fact)	NO	Graph maintenance overhead
Conservative 2PL	Pre-acquire all locks — no waiting	N/A	YES	Low concurrency, hard to predict
Wait-Die	Old waits, young dies (aborts)	NO (prevents)	YES	Younger txns may retry many times
Wound-Wait	Old wounds (aborts) younger	NO (prevents)	YES	Older txns cause younger to abort

Wait-Die vs Wound-Wait — Key Differences

Scenario	Wait-Die	Wound-Wait
Older T1 wants lock held by younger T2	T1 WAITS (old waits for young)	T1 WOUNDS T2 (aborts T2, T1 gets lock)
Younger T2 wants lock held by older T1	T2 DIES (aborts itself)	T2 WAITS (young waits for old)
Who aborts	Younger transaction always	Younger transaction always

Scenario	Wait-Die	Wound-Wait
Who initiates	Requester decides	Lock holder decides (old forces young out)
Result	Old tx never aborted	Old tx never aborted

9 & 10. Quick Revision & Interview Questions

Quick Revision

- > 2PL is a pessimistic concurrency control protocol. Enforces serializability.
- > Growing phase: ONLY acquire locks. Shrinking phase: ONLY release locks.
- > Lock Point: moment the FIRST lock is released. No new lock after this.
- > Lock Point ordering defines the serial order of transactions.
- > Basic 2PL: releases locks during shrinking. Deadlocks YES. Cascading aborts YES.
- > Conservative 2PL: all locks acquired BEFORE execution. Deadlocks NO. Cascading aborts YES.
- > Strong Strict 2PL: ALL locks held until commit/abort. Deadlocks YES. Cascading aborts NO.
- > Industry standard = Strong Strict 2PL (used by InnoDB, PostgreSQL).
- > Cascading abort: T1 releases lock early, T2 reads dirty data, T1 aborts, T2 must abort too.
- > Strong Strict 2PL prevents cascading aborts by never releasing until commit.
- > Conservative 2PL prevents deadlocks by pre-acquiring — no waiting after start.
- > Deadlock detection: Wait-For Graph (build graph, find cycle, abort victim).
- > Deadlock timeout: abort tx waiting longer than threshold. Simple but imprecise.
- > Wait-Die: Old waits, Young dies. Old tx never aborts itself.
- > Wound-Wait: Old wounds (aborts) Young. Young waits for Old.
- > Both Wait-Die and Wound-Wait: Young tx always aborts when conflict with Old tx.
- > Victim selection: abort tx with lowest cost (fewest locks, youngest, or smallest work done).
- > 2PL vs Serializable isolation: 2PL IS the mechanism; Serializable is the guarantee.

Interview Questions

- > 1. What is Two-Phase Locking? What are its two phases?
- > 2. What is the Lock Point in 2PL and why is it important?
- > 3. How does 2PL guarantee serializability?
- > 4. What are the three variants of 2PL? Compare deadlock and cascading abort risk.
- > 5. What is a cascading abort? Give an example with a money transfer.
- > 6. How does Strong Strict 2PL prevent cascading aborts?
- > 7. Why does Conservative 2PL not prevent cascading aborts?
- > 8. Explain the Wait-For Graph approach to deadlock detection.
- > 9. What is the Wait-Die scheme? What is Wound-Wait? How do they differ?
- > 10. In both Wait-Die and Wound-Wait, which transaction always aborts?
- > 11. A transaction T1(ts=1) wants a lock held by T2(ts=10). What happens in Wait-Die? In Wound-Wait?
- > 12. Which 2PL variant is used in production databases and why?
- > 13. Can Strong Strict 2PL prevent deadlocks? Explain.
- > 14. What is the tradeoff between Conservative 2PL and Strong Strict 2PL?

Key Terms

Two-Phase Locking (2PL)

Pessimistic CC protocol: growing phase (acquire) + shrinking phase (release)

Growing Phase	Acquire locks only — no releases. Lock set increases.
Shrinking Phase	Release locks only — no acquisitions. Lock set decreases.
Lock Point	First moment a lock is released. Marks transaction serialization order.
Basic 2PL	Standard 2PL. Deadlocks possible. Cascading aborts possible.
Conservative 2PL	All locks at start. No deadlocks. Cascading aborts still possible.
Strong Strict 2PL	Hold all till commit. No cascading aborts. Deadlocks still possible.
Cascading Abort	Chain of aborts caused by reading dirty (uncommitted) data.
Wait-For Graph	Directed graph of lock waits. Cycle = deadlock. Abort victim to break.
Victim	Transaction chosen to abort when resolving a deadlock.
Wait-Die	Timestamp scheme: Old waits for Young. Young dies if waits on Old.
Wound-Wait	Timestamp scheme: Old wounds (aborts) Young. Young waits for Old.
Timeout	Abort tx waiting beyond threshold. Simple deadlock resolution.
Dirty Read	Reading uncommitted data. Possible in Basic 2PL, prevented in Strong Strict.

Appendix A — Basic 2PL: Step-by-Step Examples

Basic 2PL allows locks to be released at any point during the shrinking phase — even before commit. While theoretically valid for serializability, this enables dirty reads and cascading aborts, making it impractical for real production systems.

Basic 2PL — Simple Real-World Example

Simple: Bank Transfer (T1: A->B)		
T1 acquires locks, releases mid-tx		
1	T1	X-lock on A acquired
2	T1	X-lock on B acquired
3	T1	Read A=100, Debit A: A=90
4	T1	RELEASES X-lock on A
5	T2	X-lock on A acquired (T1 released it)
6	T2	Reads A=90 (DIRTY — T1 not committed)
7	T1	Credit B: B=110, commit
8	T2	Uses A=90 in its calc, commits
Problem: Step 6: T2 read dirty A=90. If T1 aborts, T2 must cascade-abort.		

Real World: E-Commerce Inventory		
Two users buying same product simultaneously		
1	T1	X-lock: product_id=42 row
2	T1	Read stock=5, deduct: stock=4
3	T1	RELEASES X-lock on product
4	T2	X-lock: product_id=42 (T1 released)
5	T2	Reads stock=4 (T1 NOT committed)
6	T2	Places order assuming stock=4
7	T1	ABORTS — payment failed
8	T2	Must CASCADE ABORT too!
Real impact: T2 already shipped the order. Cascading abort = business logic nightmare.		

Basic 2PL — Production Verdict

Why NOT used in production:

- Cascading aborts = chain reaction of rollbacks
- Dirty reads = transactions act on uncommitted data
- Deadlocks still possible (circular lock waits)
- Recovery is complex — must track all dependent txns

When it might appear:

- + Academic / theoretical study of 2PL
- + Systems with extremely short transactions
- + When resources are NOT shared between txns
- + Basis for understanding Conservative SS-2PL

Left: bank transfer shows T2 reading dirty A=90 before T1 commits · Right: e-commerce inventory shows cascade abort chain

Simple example: T1 transfers money A->B. Releases X-lock on A mid-transaction (step 4). T2 immediately reads A=90 (dirty). T1 aborts -> T2 must cascade-abort. Balance corruption.

Real-world: E-commerce. T1 deducts inventory item, releases lock before payment confirmed. T2 reads stock=4 and places order. T1 payment fails, aborts. Stock reverts to 5 but T2 already shipped!

Production systems NEVER use Basic 2PL. It serves only as a theoretical foundation to understand why Strong Strict 2PL was developed.

Appendix B — Conservative 2PL: Step-by-Step Examples

Conservative 2PL pre-acquires ALL locks before execution begins. This completely eliminates deadlocks — no transaction ever waits mid-execution. However, the requirement to know all needed resources in advance makes it impractical for most real systems.

Conservative 2PL — Simple Real-World Example

Simple: Transfer A->B, Pre-lock All
T1 acquires ALL locks before starting

1	T1	Request X-lock on A + X-lock on B	pre-acquisition phase
2	T1	Both granted — T1 holds A and B	no other tx can touch
3	T2	Requests X-lock on A — DENIED	T1 holds A already
4	T2	WAITS for T1 to release	blocked at start
5	T1	Read A=100, Debit: A=90	executing safely
6	T1	Read B=50, Credit: B=60	executing safely
7	T1	Releases both locks	shrinking phase
8	T2	Now gets X-lock on A — proceeds	no deadlock possible

Benefit:
NO deadlock: T2 blocked at step 3. Cannot form circular wait.

Real World: Airline Seat + Meal Combo
Must lock both seat AND meal inventory upfront

1	T1	Pre-lock: seat_id=12A + meal_id=VEG	both or nothing
2	T1	Both locks granted upfront	safe to proceed
3	T2	Pre-lock: seat_id=12A + meal_id=VEG	same combo
4	T2	seat_id=12A DENIED — T1 holds it	T2 blocked at start
5	T1	Assigns seat 12A to passenger	executing
6	T1	Reserves VEG meal	executing
7	T1	Commits, releases both locks	done
8	T2	Retries — gets new available seat	clean retry

Problem:
T1 must know ALL resources needed before start — hard in complex flows.

Conservative 2PL — Production Verdict

Why NOT used in production:

- Must predict ALL required locks before execution
- Complex queries: locks depend on data values read
- Low concurrency: locks held even during idle wait
- Scheduler overhead to manage upfront lock requests

When it works well:

- + Simple, predictable transactions (fixed set of rows)
- + Deadlock-sensitive systems (safety-critical)
- + Batch jobs: known in advance which records to touch
- + Distributed txns where deadlock detection is costly

Left: bank transfer pre-locks A+B, T2 blocked at start — no circular wait possible · Right: airline combo booking locks seat+meal upfront

Simple example: T1 requests X-lock on BOTH A and B before doing any work. T2 wants A — denied immediately. T2 waits at the door. T1 executes freely, commits, releases. T2 proceeds. No deadlock ever.

Real-world: Airline booking. T1 must book seat 12A AND a VEG meal combo. Pre-locks both. T2 wants the same seat — blocked at step 3, before any execution. T1 completes the full combo booking atomically.

Why NOT in production: complex queries need locks that depend on data values read during execution (e.g. lock rows WHERE salary > X). Cannot know which rows to lock until you run the query.

Batch jobs are the closest production use: nightly reconciliation jobs that process a fixed, known set of account IDs — all IDs known upfront, pre-lock them all before starting.

Appendix C — Strong Strict 2PL: Step-by-Step Examples

Strong Strict 2PL holds ALL locks until commit or abort. This prevents dirty reads and cascading aborts completely. Deadlocks are still possible but are handled via wait-for graph detection. This is the industry standard used by InnoDB, PostgreSQL, Oracle, and SQL Server.

Strong Strict 2PL — Simple Real-World Example

Simple: Bank Transfer — Clean Execution

ALL locks held until T1 commits

1	T1	X-lock on A acquired	growing phase
2	T1	X-lock on B acquired	growing phase
3	T1	Read A=100, Debit: A=90	X-lock still held
4	T2	Tries X-lock on A — BLOCKED	T1 holds till commit
5	T1	Read B=50, Credit: B=60	X-lock still held
6	T1	COMMITTS — releases A and B locks	shrinking = immediate
7	T2	Now reads A=90 (committed data!)	no dirty read
8	T2	Proceeds safely	consistent state

Result:
T2 only ever sees committed data. No dirty reads, no cascade.

Real World: BookMyShow Seat Booking

Seat row locked until payment confirmed

1	T1	X-lock: seat_id=A1 (User Alice)	Avengers premiere
2	T1	Seat status = RESERVED	lock held
3	T2	X-lock: seat_id=A1 (User Bob) — BLOCKED	Bob must wait
4	T1	Payment processed: Rs.500 charged	still locked
5	T1	seat status = CONFIRMED	still locked
6	T1	COMMITTS — releases X-lock on A1	booking complete
7	T2	Gets lock — reads A1 = CONFIRMED	already taken
8	T2	Redirected to choose new seat	correct behaviour

Why it works:
Bob never sees 'RESERVED' state. He sees committed CONFIRMED only.

Strong Strict 2PL — Production Verdict

Why USED in production (InnoDB, PostgreSQL):

- + No cascading aborts — T2 always reads committed data
- + Strong consistency — Repeatable Read guarantee
- + Deadlock detection via wait-for graph is manageable
- + Best balance: consistency + reasonable concurrency

Known limitations:

- Deadlocks still possible: T1 holds A wants B, T2 vice versa
- Higher lock contention on hot rows (Avengers premiere!)
- Long transactions = locks held longer = others blocked
- Addressed with: TTL reservations + Optimistic CC fallback

Left: T2 blocked until T1 commits — reads clean committed A=90 · Right: BookMyShow seat A1 locked until payment confirmed

Simple example: T1 holds X-lock on A throughout. T2 tries to read A at step 4 — BLOCKED. T1 commits at step 6, releases lock. T2 now reads A=90 which is fully committed. Consistent state guaranteed.

Real-world — BookMyShow: Alice selects seat A1. X-lock acquired. Seat marked RESERVED. Bob tries to book A1 — BLOCKED until Alice completes payment. Alice pays, commits. Lock released. Bob now sees A1=CONFIRMED and is shown available seats.

Why production uses SS-2PL: No cascading aborts. No dirty reads. Wait-for graph deadlock detection is fast (milliseconds in InnoDB). Best balance between consistency and concurrency.

Deadlock scenario even in SS-2PL: Alice holds A1 wants A2. Bob holds A2 wants A1. Circular wait. InnoDB detects cycle, aborts the "cheaper" transaction (Bob), Bob retries.

Variant	Simple Example Result	Real-World Example	Production Use
Basic 2PL	T2 reads dirty A=90. If T1 aborts, cascade.	E-commerce: item shipped before payment confirmed	NEVER — cascading aborts are unacceptable
Conservative 2PL	T2 blocked at door. No deadlock. T1 runs freely.	Airline: seat+meal combo pre-locked together	RARELY — batch jobs with known resource sets only

Variant	Simple Example Result	Real-World Example	Production Use
Strong Strict 2PL	T2 blocked until T1 commits. Reads clean A=90.	BookMyShow: seat locked until payment confirmed	ALWAYS — InnoDB, PostgreSQL, Oracle standard